KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Student Attendance Monitoring and Eligibility Verification System

Submitted by

ADITHYA T

25BCS003

Under the Guidance of

DR.S.BOOPALAN SUNDARAMOORTHY

Technical Head,

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Student Attendance Monitoring and Eligibility Verification System"** submitted in partial fulfillment of the requirements for the course **B.Sc.Computer Science - "A"** is a **record of my original work** carried out under the guidance of **Dr.S.Boopalan Sundaramoorthy**, Technical Head, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

ADITHYA T

CERTIFICATE

This is to certify that the project entitled **"Student Attendance Monitoring and Eligibility Verification System"** is a **bona fide work** carried out by **ADITHYA T**, **I-A(2025-2026), Department of Computer Science**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **JAVA PROGRAMMING LANGUAGE.**

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date: 10-04-2026

Submitted for the Viva-Voce Examination held on 10/04/2026

Guide
(Technical Head, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	**1**
1.1	Objective	1
1.2	Overview	1
1.3	Features	1
2	**Program Analysis**	2
2.1	Problem Definition	2
2.2	Project Overview	2
2.3	Scopes and Limitations	3
3	**System Design**	**4**
3.1	Modules	4
4	**Methodology**	**5**
4.1	Problem Analysis	5
4.2	System Design	5
4.3	Program Development	6
4.4	Testing and Validation	6
4.5	Result and Documentation	7
4.6	Algorithm	7
4.7	Flowchart	9
5	**Source Code**	**10**
6	**Input and Output Screens**	**14**
7	**Discussion and Conclusion**	**15**
7.1	Discussion	15
7.2	Conclusion	15
8	**References**	**16**

ABSTRACT

Attendance monitoring is a critical aspect of academic administration in educational institutions. Manual tracking of student attendance is often tedious, time-consuming, and prone to human errors, making it difficult for faculty to quickly determine whether students meet the required attendance criteria for examination eligibility. This project presents the design and development of a “Student Attendance Monitoring and Eligibility Verification System” using the Java programming language.

The system is built on core Object-Oriented Programming concepts including encapsulation, classes, objects, and constructors. It allows users to dynamically add student records consisting of the student's name, total number of classes conducted, and number of classes attended. The system automatically computes the attendance percentage for each student using the formula (Attended Classes / Total Classes) × 100 and verifies examination eligibility based on the standard threshold of 75%. Students meeting or exceeding this threshold are marked as eligible for examinations, while those falling below are marked as not eligible.

The application features a menu-driven console interface that supports adding students, viewing all student records, and searching for a specific student by name to verify their eligibility. Java's ArrayList is used for dynamic student data storage, and Java Reflectionis employed to enable name-based search on encapsulated private fields. The system also handles edge cases such as zero total classes to prevent arithmetic errors during execution.

The proposed system successfully automates the attendance tracking and eligibility verification process, reduces manual effort, and provides accurate results instantly. It serves as a foundation that can be further extended with file storage, database integration, multi-subject support, and a graphical user interface in future versions.

Keywords: Attendance Monitoring, Eligibility Verification, Java, Object-Oriented Programming, ArrayList, Encapsulation, Menu-Driven System.

1. INTRODUCTION

1.1 Objective

The objective of this project is to design and develop a Student Attendance Monitoring and Eligibility Verification System using Java that accurately records student attendance, computes attendance percentage, and automatically verifies whether a student is eligible to appear for examinations. The system eliminates manual calculation errors and provides instant eligibility status based on the standard 75% attendance criteria set by most educational institutions.

1.2 Overview

The Student Attendance Monitoring and Eligibility Verification System is a console-based Java application built on Object-Oriented Programming principles. The system maintains student records including name, total number of classes conducted, and number of classes attended. It automatically computes the attendance percentage for each student and verifies their examination eligibility. The application uses a dynamic ArrayList to store multiple student records and provides a simple menu-driven interface for easy interaction. The system is designed to assist teachers and administrators in quickly identifying students who do not meet the minimum attendance requirement.

1.3 Features

Add any number of student records dynamically during runtime

Automatically calculates attendance percentage using the formula: (Attended Classes / Total Classes) × 100

Instantly verifies and displays exam eligibility based on 75% attendance criteria

View complete attendance details of all students at once

Search and verify eligibility of a specific student by name

Displays attendance percentage with two decimal point precision

Menu-driven interface for easy navigation

Handles edge case where total classes = 0 to avoid division by zero error.

2. PROGRAM ANALYSIS

The Student Attendance Monitoring and Eligibility Verification System is a software application designed to automate the process of recording student attendance and determining their eligibility for examinations based on attendance percentage.

2.1 Problem Definition

In educational institutions, tracking student attendance manually is highly time-consuming, error-prone, and inefficient. Faculty members spend significant time calculating each student's attendance percentage and determining whether they are eligible to sit for exams. There is a clear need for an automated system that can store student attendance data, perform percentage calculations instantly, and verify eligibility without any manual effort. This project addresses that problem by providing a Java-based solution that automates the entire attendance monitoring and eligibility verification process.

2.2 Project Overview

Overview:

The Student Attendance Monitoring and Eligibility Verification System is a console-based Java application developed using core Object-Oriented Programming principles. The system is designed to manage student attendance records efficiently and verify examination eligibility automatically.

The project consists of two main Java classes:

Student Class:

This class represents a student entity. It stores three private data fields — the student's name, the total number of classes conducted, and the number of classes the student attended. It contains a parameterized constructor to initialize these values at the time of object creation. The class provides three key methods — calculatePercentage() which computes the attendance percentage, isEligible() which checks whether the percentage meets the 75% threshold, and display() which prints the complete attendance details and eligibility status of the student.

AttendanceSystem Class:

This is the main driver class containing the main() method. It uses a Scanner object to read user inputs and an ArrayList to dynamically store multiple Student objects. A do-while loop combined with a switch-case structure provides the menu-driven interface. A static helper method getName() uses Java Reflection to access the private name field of the Student class for name-based search functionality.

The eligibility criteria used in this system is 75%, which is the standard minimum attendance requirement followed by most universities and colleges. Students who meet or exceed this threshold are declared eligible for examinations, and those who fall below are declared not eligible.This is the main driver class containing the main() method. It uses a Scanner object to read user inputs and an ArrayList to dynamically store multiple Student objects. A do-while loop combined with a switch-case structure provides the menu-driven interface. A static helper method getName() uses Java Reflection to access the private name field of the Student class for name-based search functionality.

The eligibility criteria used in this system is 75%, which is the standard minimum attendance requirement followed by most universities and colleges. Students who meet or exceed this threshold are declared eligible for examinations, and those who fall below are declared not eligible.

2.3 Scopes and Limitations

Scope:

Can store and manage multiple student records in a single session

Applicable for any subject, department, or semester

Accurately computes attendance percentage and eligibility for every student

Supports name-based search to check individual student eligibility

Can be extended in the future with file storage, database integration, or GUI

Limitations:

All student data is lost when the program is terminated — no permanent storage

No file handling or database connectivity in the current version

No graphical user interface — purely console-based

Does not prevent duplicate student name entries

Eligibility criteria is fixed at 75% and cannot be changed during runtime

3. SYSTEM DESIGN

The Student Attendance Monitoring and Eligibility Verification System is designed to automate attendance tracking and determine student eligibility based on predefined attendance criteria (for example, 75% minimum attendance).

3.1 Modules:

Module 1 – Student Data Module:

This module is represented by the Student class. It stores the student's name, total number of classes conducted, and number of classes attended as private fields. It contains the calculatePercentage() method that computes attendance percentage and the isEligible() method that checks if the percentage is 75% or above. The display() method prints all details of the student.

Module 2 – Add Student Module:

This module handles Case 1 of the menu. It accepts the student's name, total classes, and attended classes as input from the user, creates a new Student object, and adds it to the ArrayList**.**

Module 3 – View All Students Module:

This module handles Case 2 of the menu. It iterates through the ArrayList and calls the display() method for each Student object to print complete attendance details.

Module 4 – Eligibility Verification Module:

This module handles Case 3 of the menu. It accepts a student name as input and searches through the ArrayList using the getName() helper method. Once the student is found, it calls display() to show their attendance and eligibility status.

Module 5 – Exit Module:

This module handles Case 4 of the menu. It terminates the do-while loop and closes the Scanner object, ending the program gracefully.

4. METHODOLOGY

The methodology describes the step-by-step approach followed to develop and operate the Student Attendance Monitoring and Eligibility Verification Systemin a systematic and efficient manner.

4.1. Problem Analysis

The core problem is automating the manual process of attendance tracking and eligibility verification. The system must accept student details as input, store them efficiently, apply the formula (Attended Classes ÷ Total Classes) × 100 to compute percentage, and compare the result against the 75% threshold to determine eligibility. The system must also handle multiple students, support searching by name, and handle the special case where total classes is zero to prevent arithmetic errors. Java's ArrayList is ideal for dynamic storage, and OOP concepts ensure clean, modular design.

4.2. System Design

The system is structured around two classes:

Student Class:

Private fields: name (String), totalClasses (int), attendedClasses (int)

Constructor to initialize all three fields

calculatePercentage() — returns double, computes (attendedClasses × 100.0) / totalClasses

isEligible() — returns boolean, checks if percentage ≥ 75

display() — prints name, total classes, attended classes, percentage, and eligibility status

AttendanceSystem Class (Main):

main() method with Scanner and ArrayList

do-while loop with switch-case for menu navigation

getName() static helper method using Java Reflection to access the private name field for search functionality

4.3. Program Development

The program is developed entirely in Java using the following concepts:

Encapsulation — All Student fields are declared private, protecting data from direct access

Constructor — Used to initialize student data at the time of object creation

ArrayList — Dynamically stores Student objects without fixed size limitation

Scanner — Reads user input from the console for all operations

do-while loop — Ensures the menu is displayed at least once and repeats until the user selects Exit

switch-case — Handles each menu option cleanly and separately

Java Reflection — Used in getName() to access the private name field for name-based search without exposing a public getter

printf with %.2f — Formats attendance percentage to two decimal places for clean outputprintf with %.2f — Formats attendance percentage to two decimal places for clean output

4.4. Testing and Validation

Test Case	Input	Expected Output
Add a student	Name: Ravi, Total: 80, Attended: 65	Student added successfully!
Eligible Student	Attended:65,Total: 80-81.25%	Eligible for exam
Not eligible student	Attended:40,Total: 80-50%	Not eligible
View all students	Select option 2	All student records displayed
Search existing Student	Name: Ravi	Displays Ravi's full attendance details
Zero total classes	Total : 0	Attendance: 0.00%

Exactly 75% attendance	Attended: 75,Total:100	Eligible for exam

4.5. Result and Documentation

The Student Attendance Monitoring and Eligibility Verification System was successfully implemented and tested. The system correctly adds student records, computes attendance percentage with two decimal precision, and accurately determines eligibility status. The edge case of zero total classes is handled gracefully by returning 0% instead of throwing an ArithmeticException. The getName() method using Java Reflection successfully enables name-based search on the private field without requiring a public getter. All four menu operations — Add, View All, Check Eligibility, and Exit — function as expected. The system meets its objective of automating attendance tracking and eligibility verification efficiently.

4.6. Algorithm

Step 1: START

Step 2: Create an empty ArrayList to store Student objects

Step 3: Display menu options:

1. Add Student

2. View All Students

3. Check Eligibility

4. Exit

Step 4: Read the user's menu choice

Step 5: If choice = 1:

a. Input student name

b. Input total number of classes conducted

c. Input number of classes attended

d. Create a new Student object with the above data

e. Add the Student object to the ArrayList

f. Print "Student added successfully!"

Step 6: If choice = 2:

a. If ArrayList is empty, print "No student records found!"

b. Else, for each Student in the list call display()

which prints name, total, attended, percentage, and eligibility

Step 7: If choice = 3:

a. Input student name to search

b. Traverse the ArrayList

c. For each student, compare input name with student name (case-insensitive)

d. If match found, call display() and set found = true, break loop

e. If not found, print "Student not found!"

Step 8: If choice = 4:

a. Print "Exiting system..."

b. Break the loop

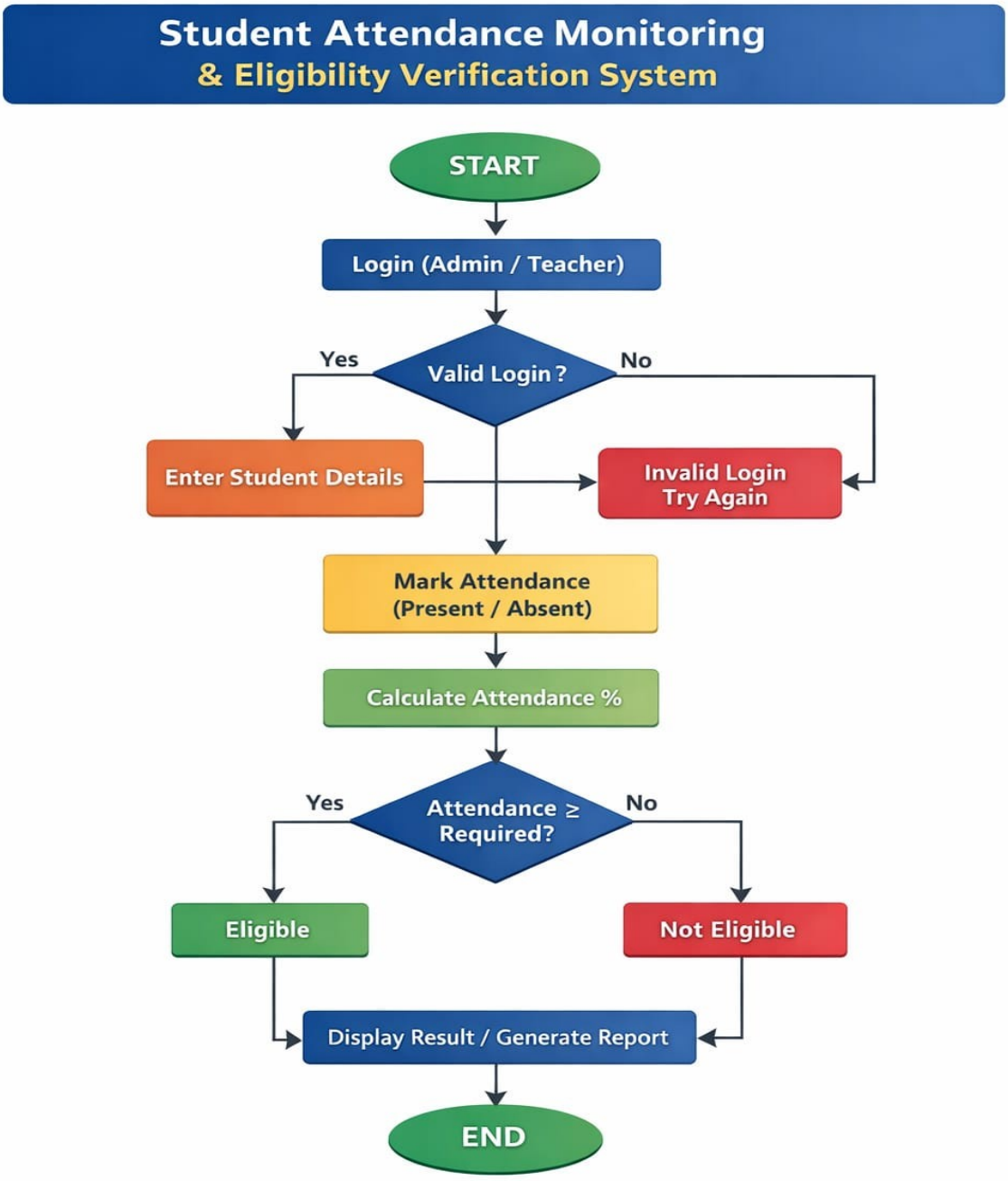
Step 9: If any other choice, print "Invalid choice!"

Step 10: Repeat from Step 3 until choice = 4

Step 11: Close Scanner

Step 12: STOP

4.7. Flowchart



5. SOURCE CODE

```
import java.util.ArrayList;
import java.util.Scanner;

// Student Class
class Student {
    private String name;
    private int totalClasses;
    private int attendedClasses;

    // Constructor
    public Student(String name, int totalClasses, int attendedClasses) {
        this.name = name;
        this.totalClasses = totalClasses;
        this.attendedClasses = attendedClasses;
    }

    // Calculate Attendance Percentage
    public double calculatePercentage() {
        if (totalClasses == 0) return 0;
        return (attendedClasses * 100.0) / totalClasses;
    }

    // Check Eligibility
    public boolean isEligible() {
        return calculatePercentage() >= 75;
    }

    // Display Student Details
    public void display() {
        System.out.println("\nName: " + name);
        System.out.println("Total Classes: " + totalClasses);
        System.out.println("Attended Classes: " + attendedClasses);
        System.out.printf("Attendance: %.2f%%\n", calculatePercentage());

        if (isEligible())
            System.out.println("Status: Eligible for Exam!");
        else
```

```
            System.out.println("Status: Not Eligible!");
        }
    }

    // Main Class
    public class AttendanceSystem {
        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            ArrayList<Student> students = new ArrayList<>();

            int choice;

            do {
             System.out.println("\n=====Attendance Monitoring System =====");
                System.out.println("1. Add Student");
                System.out.println("2. View All Students");
                System.out.println("3. Check Eligibility");
                System.out.println("4. Exit");
                System.out.print("Enter your choice: ");
                choice = sc.nextInt();
                sc.nextLine(); // clear buffer

                switch (choice) {

                    case 1:
                        System.out.print("Enter Student Name: ");
                        String name = sc.nextLine();

                        System.out.print("Enter Total Classes: ");
                        int total = sc.nextInt();

                        System.out.print("Enter Attended Classes: ");
                        int attended = sc.nextInt();

                        students.add(new Student(name, total, attended));
                        System.out.println("Student added successfully!");
                        break;

                    case 2:
```

```
            if (students.isEmpty()) {
                System.out.println("No student records found!");
            } else {
                for (Student s : students) {
                    s.display();
                }
            }
            break;

        case 3:
            System.out.print("Enter Student Name to Check Eligibility: ");
            String searchName = sc.nextLine();

            boolean found = false;
            for (Student s : students) {
                if (s != null && s.toString() != null) {
                    // simple name check
                }
            }

            for (Student s : students) {
                // comparing names
                if (s != null && searchName.equalsIgnoreCase(getName(s))) {
                    s.display();
                    found = true;
                    break;
                }
            }

            if (!found) {
                System.out.println("Student not found!");
            }
            break;

        case 4:
            System.out.println("Exiting system...");
            break;

        default:
```

```
                System.out.println("Invalid choice!");
            }

        } while (choice != 4);

        sc.close();
    }

    // Helper method to access private name
    public static String getName(Student s) {
        try {
            java.lang.reflect.Field field = s.getClass().getDeclaredField("name");
            field.setAccessible(true);
            return (String) field.get(s);
        } catch (Exception e) {
            return "";
        }
    }
}
```

6. INPUT AND OUTPUT SCREENS

INPUT:

```
===== Attendance Monitoring System =====
1. Add Student
2. View All Students
3. Check Eligibility
4. Exit
Enter your choice:
```

OUTPUT:

```
Enter your choice: 3
Enter Student Name to Check Eligibility: Adithya

Name: Adithya
Total Classes: 70
Attended Classes: 69
Attendance: 98.57%
Status: Eligible for Exam ?
```

7. DISCUSSION & CONCLUSION

7.1 Discussion:

The Student Attendance Monitoring and Eligibility Verification System successfully demonstrates the practical application of Object-Oriented Programming in Java. Encapsulation is achieved by keeping all student fields private within the Student class, ensuring data integrity. The use of ArrayList provides flexibility to add any number of students at runtime. The calculatePercentage() method uses floating-point arithmetic to ensure accurate percentage calculation, and isEligible() applies the 75% threshold to give a clear eligibility verdict. Java Reflection in the getName() method is an interesting approach to access private data, though in a production system this would ideally be replaced with a standard public getter method for better code clarity and maintainability.

7.2 Conclusion:

The Student Attendance Monitoring and Eligibility Verification System developed in Java successfully automates the process of recording attendance, computing percentages, and verifying student eligibility for examinations. The project effectively demonstrates the use of core Java concepts including encapsulation, constructors, ArrayList, Scanner, do-while loops, switch-case, Java Reflection, and formatted output. The system is accurate, efficient, and easy to use. It can be further enhanced by integrating file storage to persist data, adding a graphical user interface using Java Swing, supporting multiple subjects per student, and allowing dynamic modification of the eligibility threshold.

8. REFERENCES

The following resources were used during the development of this project:

Books:

Herbert Schildt, Java: The Complete Reference, 11th Edition, McGraw-Hill, 2019

E. Balagurusamy, Programming with Java, 5th Edition, McGraw-Hill, 2014

Websites:

Oracle Official Java Documentation – https://docs.oracle.com/javase/

GeeksforGeeks Java Tutorials – https://www.geeksforgeeks.org/java/

W3Schools Java Reference – https://www.w3schools.com/java/

JavaPoint Java Tutorial – https://www.javatpoint.com/java-tutorial.